08 線性時間排序

Part II Sorting and Order Statistics

本章定位

線性排序以鍵值表示或輸入分布換取速度，必須先分清比較模型的下界。本章把counting的區塊位置、LSD的穩定性與bucket的平方桶大小逐一推導，並明列鍵範圍、位元數、分布及空間成本。

前置知識

比較決策樹、整數鍵值範圍與穩定排序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 證明比較排序的 Omega(n log n) 下界。
- 分析 counting sort 的適用條件與穩定版本。
- 理解 radix sort 與 bucket sort 的成本假設。
- 8.1 Lower bounds for sorting
- 8.2 Counting sort
- 8.3 Radix sort
- 8.4 Bucket sort

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

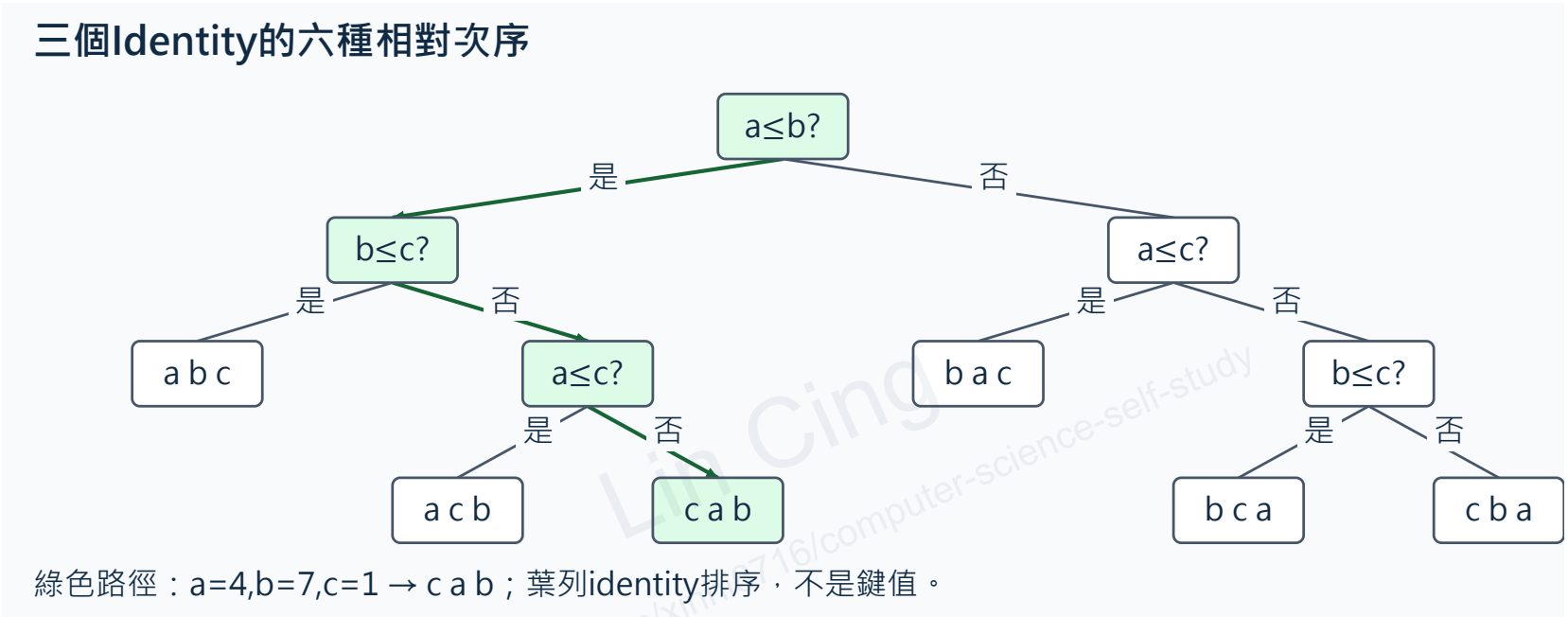
8.1 Lower bounds for sorting

比較排序只能透過兩鍵的比較結果取得次序資訊，不能讀鍵值當索引。本節先限制n個相異鍵，於是每次≤比較只有兩個可能分支。固定n與演算法後，內節點標比較的兩個原始identity，根到葉記錄一次執行；葉必須決定完整的identity排序。

n!種相對次序必須導向可區分的葉：若兩個不同次序到同一葉，演算法在相同結果序列後只能輸出同一個identity排列，至少一個輸入會錯。即使演算法有多個葉代表同一排序，也至少需n!個可達葉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.1 Lower bounds for sorting （續）



圖的每片葉標的是identity順序，例如「c a b」表示c<a<b，不是演算法可任意輸出的六組數值。a=4、b=7、c=1走左、右、右，得到c a b。

最壞下界的量詞與模型

高度h的二元樹至多2^h葉，故h≥⌈log₂(n!)⌉。不用Stirling也能求同階界：n!≤n^n，而最後⌈n/2⌉個因子各至少n/2，所以

(n/2)^(n/2) ≤ n! ≤ n^n，因此log₂(n!)=Θ(n log n)。

結論是：對每個正確的比較排序、每個n，至少存在一個長度n輸入需要Ω(n log n)次比較。它不是「每個輸入都需n log n」，例如insertion sort遇已排序資料只需n−1次鍵比較。

相異鍵只是取出所有輸入中的一個困難子集，所以允許重複鍵不會降低一般排序的最壞下界。Count/radix利用整數表示和陣列索引，已離開比較模型，不與定理矛盾；同時也多了承受鍵範圍與位元數的成本。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

平均比較數也需要證明

若n!種相對次序均勻，選每種次序的一片可達葉，深度為$d_1,\cdots,d_N$，$N=n!$。不同葉的根路徑是prefix-free二元字串，故Kraft不等式$\Sigma 2^{-d_i}\le 1$；可由把深度d葉視為[0,1)中長度2^{-d}的互不相交二元區間證明。

算術平均≥幾何平均給`2^(-(Σd_i)/N) ≤ (Σ2^(-d_i))/N ≤ 1/N`，取log得平均深度$\ge \log_2 N=\Omega(n \log n)$。不是只用最高葉深推出平均深度。

對每次都正確的隨機比較排序，固定內部亂數得到一棵正確決策樹；對均勻輸入平均後，每棵樹仍有此下界，再對亂數取期望。故至少一個輸入的期望比較數也達$\Omega(n \log n)$。這裡沒有處理允許錯誤輸出的Monte Carlo排序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.2 Counting sort

鍵是0..k的整數，記錄另含satellite data（隨鍵搬移的其他欄位）。本節輸入/輸出位置採1-index，計數表鍵採0-index。

```
C[0..k]=0
for j=1..n: C[key(A[j])]++
for v=1..k: C[v]+=C[v-1]
for j=n downto 1:
    v=key(A[j])
    B[C[v]]=A[j]
    C[v]--
return B
```

計數後C[v]=該鍵頻率；前綴和後C[v]=鍵≤v的記錄數，也就是v區塊最後一格。回填過程會修改C；若還需要範圍查詢，須先保存未消耗的前綴表P。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

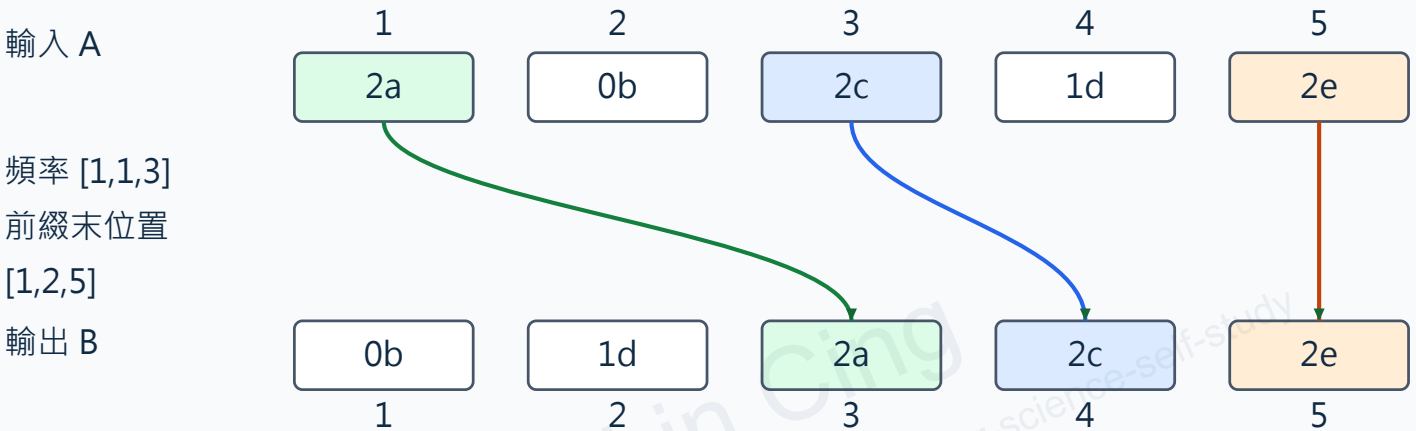
從頻率到實際輸出位置

原創例A=[2a,0b,2c,1d,2e]，字母是identity。頻率C=[1,1,3]，累積末位置[1,2,5]。三個區塊為鍵0的位置1、鍵1的位置2、鍵2的位置3..5。

掃描j	記錄	放入B位置	放完後C
5	2e	5	[1,2,4]
4	1d	2	[1,1,4]
3	2c	4	[1,1,3]
2	0b	1	[0,1,3]
1	2a	3	[0,1,2]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Counting：追蹤同鍵Identity的目的位置



反向處理 e→d→c→b→a；放入位置 5→2→4→1→3。

綠a、藍c、橘e是三筆鍵2的identity；輸出仍保持 a→c→e。

輸出[0b,1d,2a,2c,2e]。不能只看裸鍵排序就宣稱穩定，必須追蹤a,c,e的相對次序。

回填不變量、穩定性與成本

固定鍵v，令L=原始P[v−1]（v=0時L=0）、R=P[v]。處理某個suffix後，該suffix內鍵v的記錄已佔區塊右端；C[v]正指向尚未填滿區塊的最右位置。反向遇到下一個v，就填C[v]並減一，不重覆、不越出[L+1,R]。處理完全部記錄，各鍵恰填滿其頻率大小的區塊，區塊依鍵遞增排列，故正確。

若同鍵記錄x在輸入先於y，反向掃描先放y於較右格，再放x於較左格，所以輸出x仍先於y。若沿用「末位置後減一」卻改正向掃描，鍵排序仍正確，但同鍵identity反轉。正向也可穩定：改用每個鍵的「起始位置後加一」，兩種規格不能混搭。

四段迴圈時間Θ(n+k+1)，額外空間Θ(n+k+1)，包含輸出B；不是原地排序。鍵可負時先用v−min平移，但範圍k=max−min仍可能巨大；讀寫整個鍵及索引是本章word-RAM的一步假設。

保存P後，0≤a≤b≤k的閉區間計數為P[b]−P[a−1]，約定P[−1]=0。例[1,2]含P[2]−P[0]=5−1=4筆；查詢O(1)，預處理Θ(n+k+1)。

8.3 Radix sort

對固定d位、每位0..r−1的非負整數，LSD（least-significant digit，最低有效位優先）由低位到高位，每輪對目前位做穩定排序。

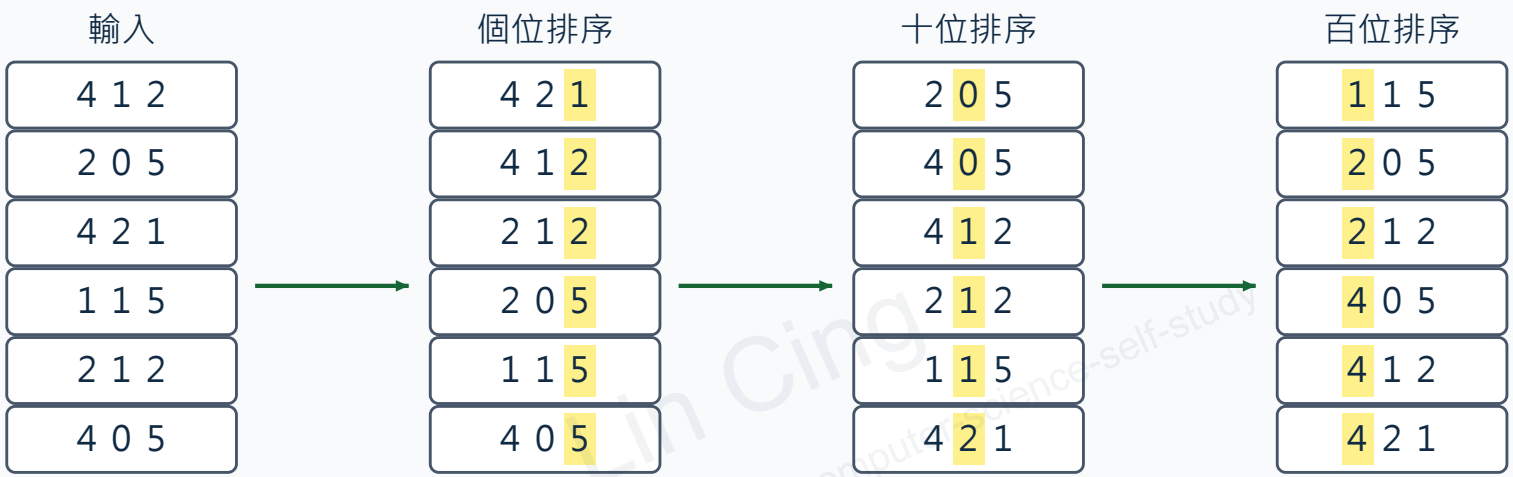
歸納不變量：完成第t輪後，整個序列已按最低t位的數值排序。t=0沒有條件；第t輪先依第t位分組，不同組的第t位決定大小。同組內靠穩定性保留前t−1位的排序，故合併成最低t位的完整次序。t=d後就是完整鍵序。

「下一輪保留低位排序」只適用同一高位組，不能誤讀成輸出依然全域按低位遞增。位排序不穩定時，這個歸納步失效；例如[12,11]先個位成[11,12]，十位同為1若反轉回[12,11]就錯。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

三輪算例與相同位數的次序

LSD Radix：低位順序成為高位Tie-break



黃底是本輪排序位；同位值相同的記錄保留上一欄順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三輪算例與相同位數的次序 （續）

輪次	完整序列
輸入	[412,205,421,115,212,405]
個位	[421,412,212,205,115,405]
十位	[205,405,412,212,115,421]
百位	[115,205,212,405,412,421]

十位輪的205、405同為0，必保留前輪的205在405之前；412、212、115同為1，依前輪順序保留。百位輪中205在212之前是已排序低兩位05<12的結果。

把整個陣列反過來由百位→十位→個位排序，即使每輪穩定，也可能讓低位成為最後的主序。正確的MSD方法要在相同高位群內遞迴，不能把兩種演算法混為一談。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

位元數、radix與成本的共同分母

每輪穩定counting sort的時間$\Theta(n+r)$，d輪共$\Theta(d(n+r))$，額外空間$\Theta(n+r)$。d為常數且$r=O(n)$才直接得到$\Theta(n)$；鍵若長到無法常數時間取位，還要計位元存取成本。

b位元鍵每輪取s位（$1 \le s \le b$），則$r=2^s$、$d=\lceil b/s \rceil$，成本$\Theta(\lceil b/s \rceil(n+2^s))$。最高不足$s$位可補0，仍可用同一表。$n \ge 2$時選$s=\min(b,\lfloor \log_2 n \rfloor)$，有$2^s \le n$，得到$O(n\lceil b/\min(b,\lfloor \log_2 n \rfloor)\rceil)$；若$b=O(\log n)$就是$O(n)$。

以「每輪$n+r$單位」這個簡化模型比較32位元鍵：$s=8$需$4(n+256)$，$s=16$需$2(n+65536)$。後者較小當且僅當$n>65024$，$n=65024$時相同。這些係數是題設模型，不能當成實測時間；cache、配置、記憶體頻寬會改變實機選擇。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8.4 Bucket sort

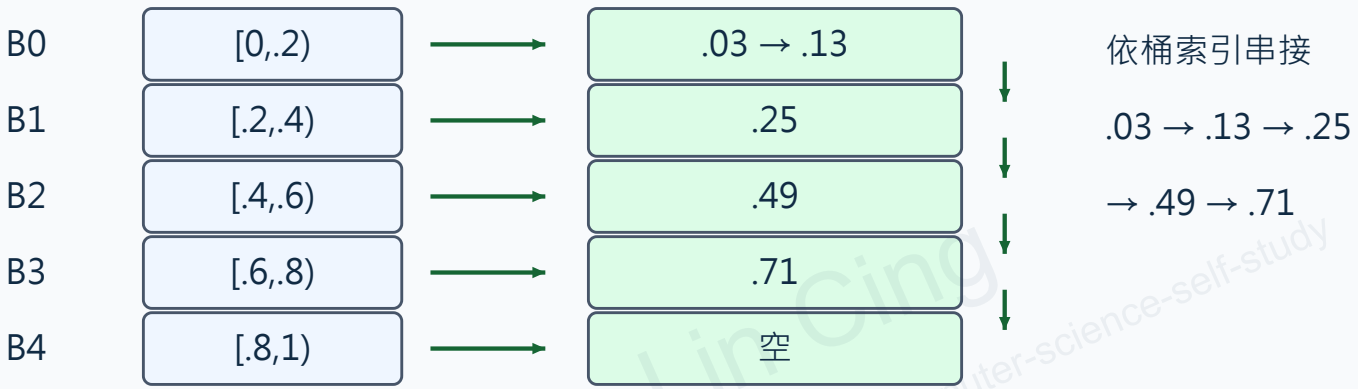
輸入n個實數$x \in [0,1)$，$n \geq 1$，建立n個桶$B[0..n-1]$，放入$B[\lfloor nx \rfloor]$；每桶用insertion sort排序，再依桶索引串接。$n=0$直接回空序列，不計$\lfloor nx \rfloor$。$x=1$不在定義域，不能拿它索引$B[n]$。

正確性不依賴分布：若$x \leq y$，則$\lfloor nx \rfloor \leq \lfloor ny \rfloor$。不同桶的所有元素已按區間有序，同桶由內部排序完成，所以串接即全排序。每桶按輸入順序append，再用穩定insertion sort，才也保留同鍵identity次序；頭插會先改變順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bucket：先由區間保證跨桶次序

輸入 .13,.25,.03,.71,.49 → 索引 0,1,0,3,2



藍：桶的半開區間；綠：桶內排序結果；跨桶無需再比較。

例n=5，輸入[.13,.25,.03,.71,.49]，桶索引依序[0,1,0,3,2]，不是[0,0,0,1,2]。B0排序為[.03,.13]，B1=[.25]、B2=[.49]、B3=[.71]、B4空；輸出[.03,.13,.25,.49,.71]。

線性期望要計算平方桶大小

設N_i為第i桶大小。建立、分配、串接共$\Theta(n)$，桶內insertion sort總成本$O(\Sigma N_i^2)$，所以$T=O(n+\Sigma N_i^2)$。僅知$E[N_i]=1$不足以把$E[N_i^2]$替換成1。

假設各輸入獨立且均勻[0,1)。X_{ij}表示第j筆落桶i，$N_i=\Sigma_j X_{ij}$。$X_{ij}^2=X_{ij}$，$E[X_{ij}]=1/n$；$j\neq k$時，輸入獨立給$E[X_{ij}X_{ik}]=1/n^2$。因此

$E[N_i^2]=\Sigma_j 1/n+\Sigma_{(j\neq k)}1/n^2=1+n(n-1)/n^2=2-1/n$。

總和$E[\Sigma N_i^2]=2n-1$，故$E[T]=O(n)$，而讀取n筆需$\Omega(n)$，合為$\Theta(n)$。這裡用的是不同記錄間的獨立性；不同桶的N_i彼此反而相關，因$\Sigma N_i=n$，但期望線性性不要求它們獨立。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

分布、相關性與最壞情況

若各記錄獨立但桶機率為p_i，$\Sigma p_i=1$，同樣展開得

$E[\Sigma N_i^2]=n+n(n-1)\Sigma p_i^2$。

均勻時$\Sigma p_i^2=1/n$。只要此和為$O(1/n)$，仍有線性期望；若單桶機率固定為1，平方成本就是n^2。分布越集中，撞桶的成對機率Σp_i^2越高。

邊際均勻仍不夠：先均勻抽$J\in\{0,\dots,n-1\}$，令所有記錄$x_j=(J+1/2)/n$。每筆落每桶的邊際機率都$1/n$，$E[N_i]=1$，但每次所有n筆同桶，所以$\Sigma N_i^2=n^2$。此反例針對平方成本界，並不宣稱相等鍵的穩定insertion sort本身要平方時間。

真實最壞時間的達界族可取$n\geq 2$、$x_j=(n-j+1)/(2n^2)$，$j=1..n$：全部在第一桶且嚴格遞減，insertion sort搬移$n(n-1)/2$次，故$\Theta(n^2)$。把桶內排序換成$O(m\log(m+1))$最壞演算法，可令總最壞$O(n\log n)$，並保留原獨立均勻模型的線性期望，因$m\log(m+1)=O(m^2)$。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 08。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study